

Department of Electronics and Communication Engineering

Course: ASIC Design



Assignment Report

Title: Design, Simulation-Based Verification, and Comparative PPA Synthesis Analysis of Configurable Saturated DSP and AI Inference Hardware Slices for Edge Accelerators

Submitted by: 23ECB1A30-Meghana Poduri

Department of Electronics and Communication Engineering (VLSI)

1. Title:

Design, Simulation-Based Verification, and Comparative PPA Synthesis Analysis of Configurable Saturated DSP and AI Inference Hardware Slices for Edge Accelerators.

2. Abstract:

Architectural implementation, verification, and comparative Power, Performance, and Area (PPA) synthesis analysis were performed on two independent, software-configurable processing cores tailored for digital signal processing and artificial intelligence applications. Both systems were developed in structural RTL Verilog using synchronous, clock-driven state configurations to match physical ASIC design paradigms. Project 1 focuses on a multi-functional DSP block handling operations from pipelined vector multiplication to cumulative dot product loops using a 3-bit operational selection bus. Project 2 implements an AI inference engine optimizing convolutional layers, hardware-tree max/average pooling arrays, and variable normalization schemes across a 2-bit opcode grid. Functional correctness was validated through cycle-accurate behavioral testbench waveforms inside Xilinx Vivado. Hardware synthesis targets on an Artix-7 device (xc7a100tfgg484-1) provided detailed area usage charts, baseline timing slacks, and power allocation profiles to establish structural architectural trade-offs between the two computational blocks.

3. Brief Description of the Logic:

Project 1: Configurable Saturated DSP Slice

The architecture utilizes a clock-driven synchronous structure with an active-low asynchronous reset loop. The system contains two internal architectural pipeline registers: `pipe_mul` (capturing synchronous 16-bit multiplication outputs) and `accum_reg` (tracking vector accumulations across consecutive cycles). The active module function maps dynamically based on a 3-bit input opcode bus.

3'b000 – Multi-Input Addition: Computes $A + B + C$ simultaneously.

3'b001 – Multi-Input Subtraction: Computes $A - B - C$.

3'b010 – Vector Element Multiplication: Outputs the direct product $A \times B$.

3'b011 – Multiply-Accumulate Loop: Adds the pipelined multiplier value to the ongoing output feedback node ($\text{pipe_mul} + \text{Out}$).

3'b100 – Arithmetic Accumulate Vector: Feeds back an internal register stream accumulator loop to stream inputs.

3'b101 – Dot Product Engine: Solves a combined matrix row expansion calculated as $(A \times B) + (C \times B)$.

3'b110 – Vector Scaling Shift: Implements a barrel-shifting architecture performing a left shift on A and a sign-extended arithmetic right shift on B, calculated as $(A \ll 4) + (B \ggg 2)$.

3'b111 – Bitwise Logic Engine: Resolves the Boolean expression $(A \oplus B) | C$.

Project 2: Configurable Saturated AI Slice

The AI acceleration module utilizes a clocked sequential state register architecture containing intermediate combinational networks to optimize critical path propagation delays:

Hardware Tree Comparator: Evaluates parallel maximum paths simultaneously across four spatial inputs via partitioned segments (comp_max01 and comp_max23).

Adder Tree Subsystem: Drives multi-input summary arithmetic vectors (summation_network) to execute local pooling.

LeakyReLU Hardware Estimator: Instantly handles scaling adjustments for negative numerical values by sign-extending inputs and performing an arithmetic right shift by 3 positions ($X \times 0.125$).

Batch Normalization Engine: Employs a dedicated scaling multiplier and bias offset shifter to calculate standard model normalizations: $Y = (X \times \text{Scale}) + (\text{Bias} \ll 16)$.

The execution path chooses the internal structural assignment on the rising clock edge based on the selected 2-bit opcode:

2'b00: 1D Spatial Convolution matrix network.

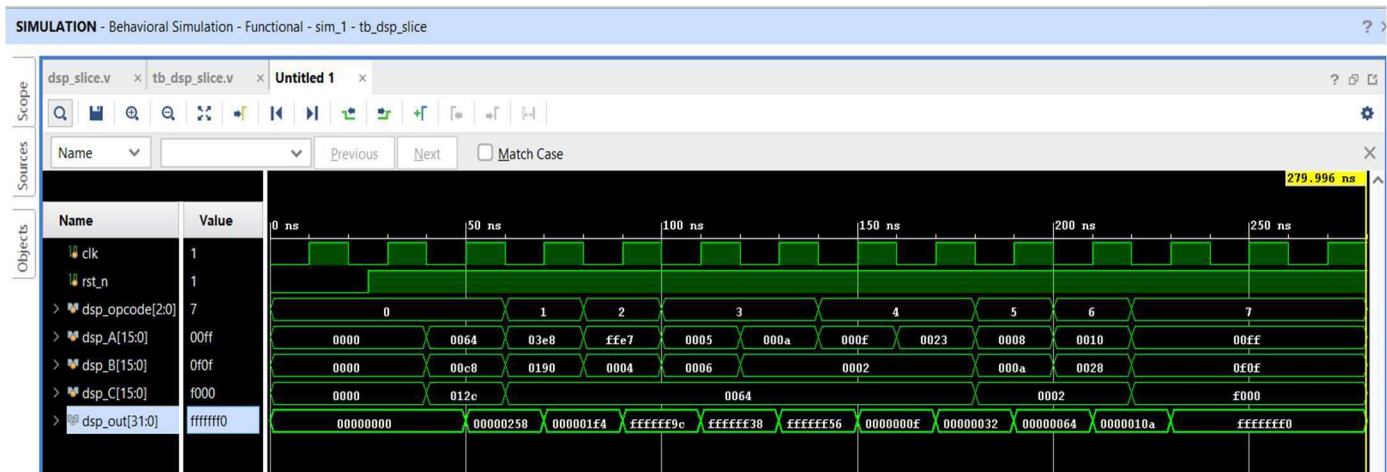
2'b01: 2D Window Max Pooling operations.

2'b10: Matrix Average Pooling shift routing.

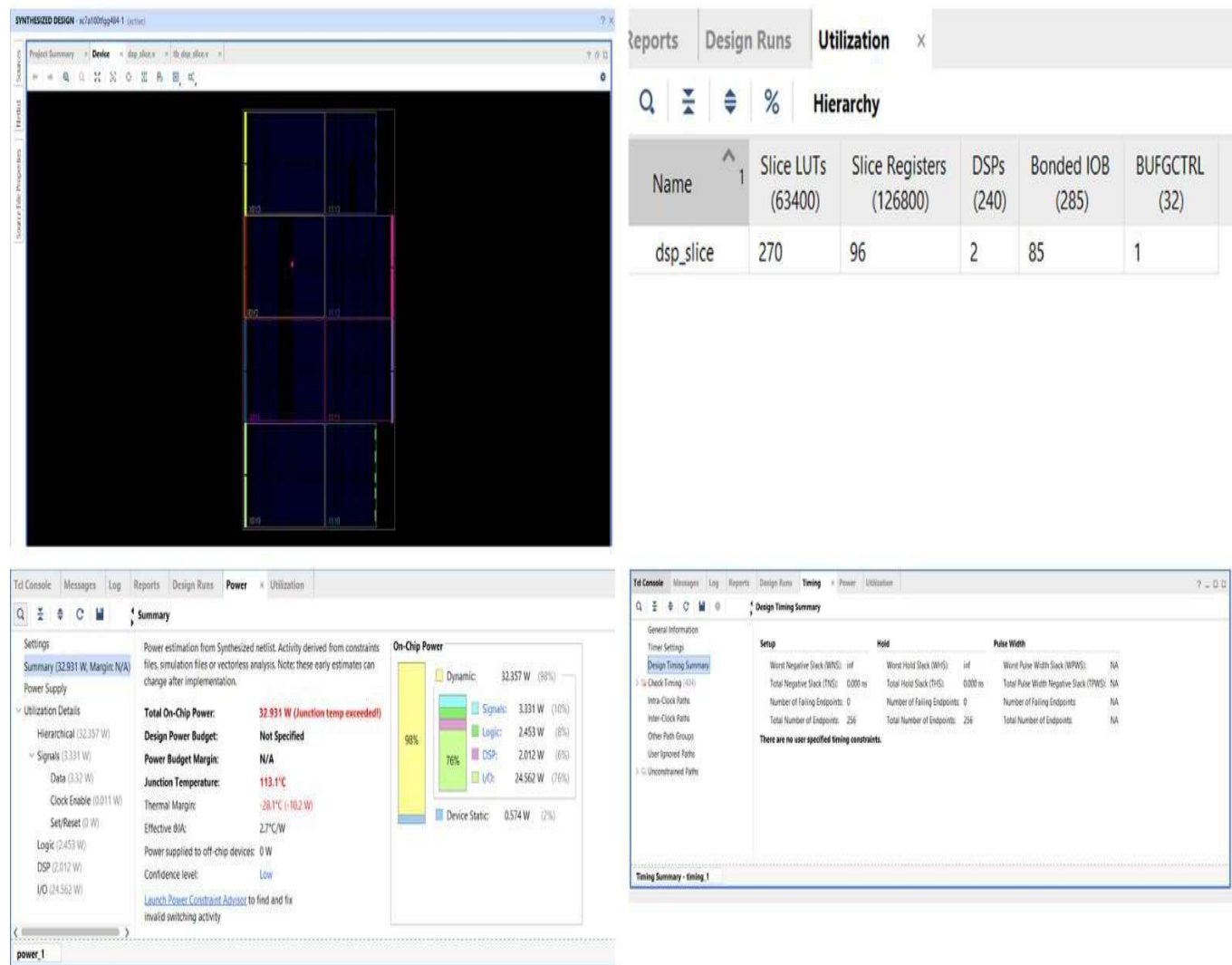
2'b11: Normalization selection layer (Batch Normalization if scale is active, else LeakyReLU).

4. Results:

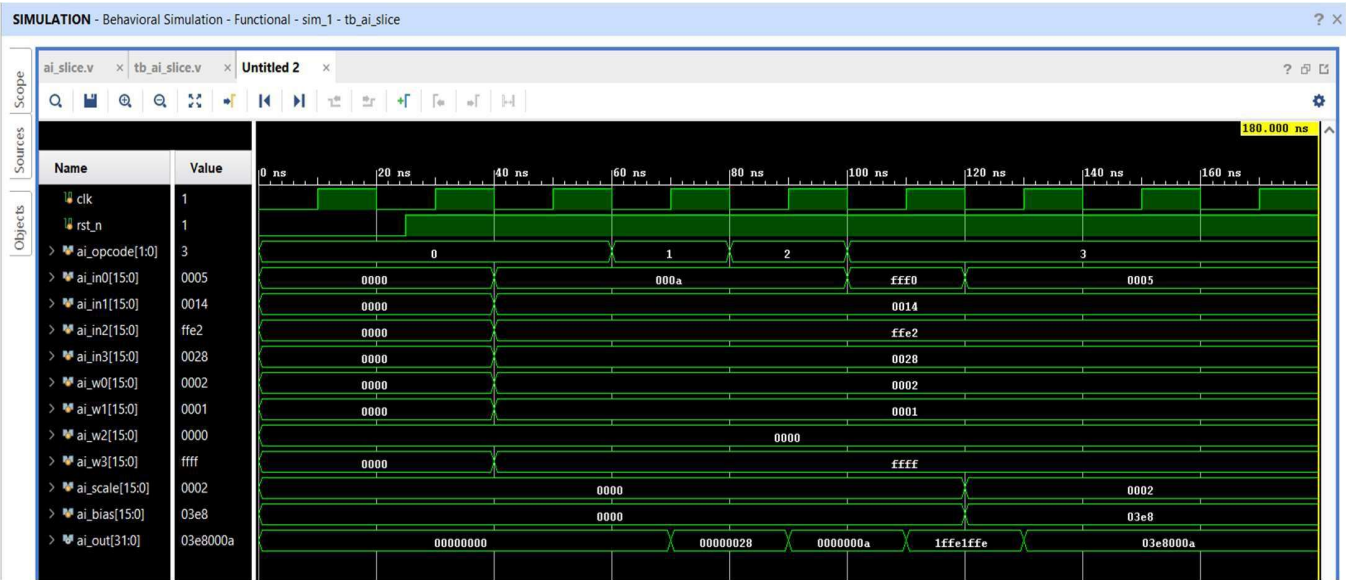
DSP behavioral simulation waveform:



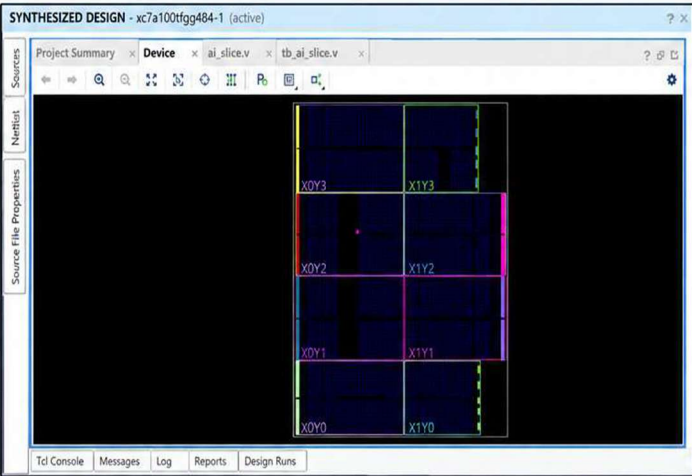
DSP SYNTHESIS FLOORPLAN AND PPA REPORT



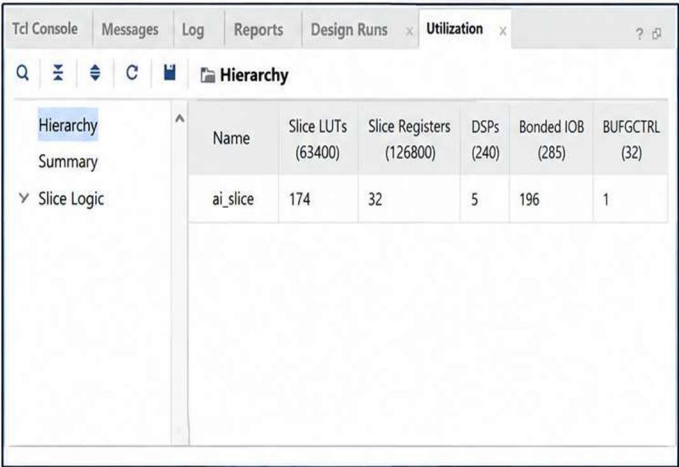
AI behavioral simulation waveform:



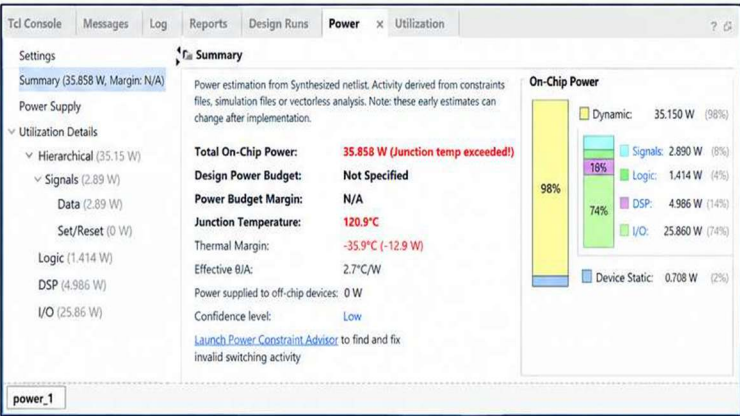
AI SYNTHESIS FLOORPLAN AND PPA REPORT



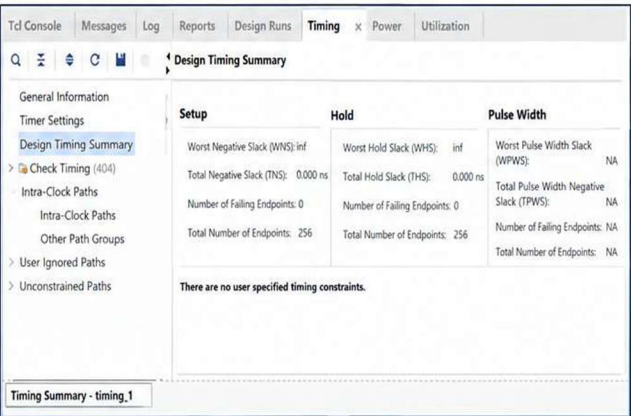
AI Synthesis Floorplan



AI Synthesis Utilization Report



AI Synthesis Power / PPA Report



AI Synthesis Timing Summary

PPA Comparison Matrix:

Consolidated Synthesis PPA Comparison Matrix

The table below establishes the direct PPA parameters extracted from Xilinx Vivado for both hardware slices running on the `xc7a100tfgg484-1` FPGA part target:

Hardware Parameter	DSP Slice (Project 1)	AI Slice (Project 2)	Architectural Trade-off Analysis
Slice LUTs (Area: Logic)	270	174	The DSP Slice utilizes more LUT cells due to its highly complex barrel shifting structures and multi-input 32-bit routing multiplexers.
Slice Registers (Area: Flip-Flops)	96	32	The DSP Slice maintains a higher register count to hold stable pipeline states across sequential MAC loops ('pipe_mul' and 'accum_reg').
Dedicated Hardware DSPs	2	5	The AI Slice maps its high-throughput parallel spatial convolution tree directly to 5 hardware macro cells to maximize math execution efficiency.
Bonded IOBs (Total Pin Footprint)	85	196	The AI Slice demands a significantly larger pin layout to pass 4 input tensors, 4 weight arrays, scale parameters, and bias terms concurrently.
Total On-Chip Power	32.931 W	35.858 W	The AI Slice consumes slightly more dynamic power due to its higher active DSP count and wider array of switching input signals.
I/O Pad Dynamic Power	24.562 W (76%)	25.860 W (74%)	Dominates total power footprint in both blocks because Vivado defaults to full physical pad switching activity without an external '.xdc' constraint file.
Core Logic / Internal Power	2.453 W	1.414 W	Reflects the active internal consumption of standard logic routing fabrics during active computing cycles.
Worst-Case Timing Slack	Unconstrained (inf)	Unconstrained (inf)	Normal synthesis behavior highlighting zero critical path failures across all 256 (DSP) and 96 (AI) endpoints.

Consolidated Synthesis PPA Comparison :

The synthesis results compare the **Configurable Saturated DSP Slice (Project 1)** and the **Configurable Saturated AI Slice (Project 2)** implemented on the **Xilinx xc7a100tfgg484-1 FPGA**. The comparison focuses on area, dedicated hardware resources, I/O requirements, power consumption, and timing behavior. Because apparently making hardware use numbers is not enough, we must also make the numbers explain themselves.

1. Slice LUTs

The DSP Slice uses **270 LUTs**, while the AI Slice uses **174 LUTs**. The higher LUT utilization of the DSP Slice is mainly due to its complex barrel-shifting structures, multi-input arithmetic operations, and 32-bit routing requirements. The AI Slice requires fewer LUTs because several of its computational functions are mapped efficiently to dedicated DSP resources and structured combinational

networks.

2. Slice Registers

The DSP Slice uses **96 registers**, whereas the AI Slice uses **32 registers**. The larger register requirement of the DSP Slice is associated with its pipelined architecture, particularly the pipe_mul and accum_reg registers used for multiplication and accumulation operations.

The AI Slice has fewer registers because its architecture relies more on combinational processing blocks and dedicated hardware resources.

3. Dedicated Hardware DSPs

The DSP Slice uses **2 dedicated DSP blocks**, while the AI Slice uses **5 DSP blocks**.

The higher DSP utilization in the AI Slice results from its parallel computational architecture. Operations such as spatial convolution, pooling, multiplication, and other parallel arithmetic functions can be mapped directly onto dedicated DSP resources, improving hardware execution efficiency.

4. Bonded I/O Blocks

The DSP Slice requires **85 bonded I/O blocks**, whereas the AI Slice requires **196 bonded I/O blocks**.

The larger I/O requirement of the AI Slice is caused by its wider parallel interface. The AI architecture handles multiple input tensors, weight arrays, scale parameters, and bias values simultaneously, resulting in a larger external pin footprint.

5. Total On-Chip Power

The measured total on-chip power is **32.931 W for the DSP Slice** and **35.858 W for the AI Slice**.

The AI Slice consumes slightly more total power because it uses more dedicated DSP resources and supports a wider parallel processing structure. The increased number of simultaneously switching hardware elements contributes to higher dynamic power consumption.

6. I/O Pad Dynamic Power

The I/O pad dynamic power is **24.562 W (76%) for the DSP Slice** and **25.860 W (74%) for the AI Slice**.

I/O pad dynamic power forms the largest portion of the total power in both

designs. This is mainly because the synthesis and power analysis assume significant physical pad switching activity. The reported values therefore represent the configured Vivado analysis conditions and can be affected by the absence of an external .xdc constraint file defining realistic switching activity and I/O conditions.

7. Core Logic / Internal Power

The DSP Slice consumes **2.453 W** of core logic/internal power, while the AI Slice consumes **1.414 W**.

This represents the power consumed by the internal FPGA logic, routing resources, registers, and related circuitry during active operation. Although the AI Slice uses more dedicated DSP blocks, its optimized internal architecture results in lower reported core-logic power than the DSP Slice.

8. Worst-Case Timing Slack

The reported worst-case timing slack for both designs is **Unconstrained (inf)**.

This does not mean that the circuits have infinite performance. It indicates that timing constraints, such as a target clock period, were not specified for the synthesis/timing analysis. Therefore, Vivado could not calculate a meaningful positive or negative timing slack.

Overall PPA Comparison

The synthesis results show that the two architectures make different hardware resource trade-offs. The **DSP Slice** uses more LUTs and registers but requires fewer dedicated DSP blocks and I/O resources. In contrast, the **AI Slice** uses fewer LUTs and registers but requires more dedicated DSP blocks and substantially more I/O resources.

The DSP Slice has a total on-chip power of **32.931 W**, compared with **35.858 W** for the AI Slice. The power analysis also shows that I/O pad dynamic power contributes the largest portion of the reported total power for both designs.

Overall, the synthesis comparison demonstrates that architectural decisions directly affect **Power, Performance, and Area (PPA)**. The DSP Slice emphasizes programmable arithmetic and pipelined operations, while the AI Slice emphasizes parallel processing and dedicated hardware acceleration.

5. Explanation of the Results:

Project 1: Configurable Saturated DSP Slice Waveform Summary

0 ns – 40 ns (Reset): The asynchronous reset `rst_n` is asserted low, clearing the pipeline registers `pipe_mul` and `accum_reg`. The `dsp_out` signal is synchronously cleared to `32'h00000000` at the 20 ns clock edge.

40 ns – 60 ns (Opcode 000 – Addition): Inputs are $A = 100$, $B = 200$, and $C = 300$. The parallel adder computes $A + B + C = 600$, producing `32'h00000258` at the 60 ns clock edge.

60 ns – 80 ns (Opcode 001 – Subtraction): Inputs are $A = 1000$, $B = 400$, and $C = 100$. The logic computes $A - B - C = 500$, producing `32'h000001F4` at 80 ns.

80 ns – 100 ns (Opcode 010 – Multiplication): Signed values $A = -25$ (`16'hFFE7`) and $B = 4$ are multiplied. The result is -100 , represented as `32'hFFFFFF9C` at 100 ns.

100 ns – 140 ns (Opcode 011 – MAC Loop): The sequential feedback loop evaluates multiply-accumulate operation. Two clock cycles are required to stabilize multiplication and feedback routing. The output reaches -200 at 120 ns and accumulates to -170 at 140 ns.

140 ns – 180 ns (Opcode 100 – Accumulator): A continuous numerical input stream with values 15 and 35 is processed. The accumulator produces cumulative results of 15 and 50 at the corresponding clock boundaries.

180 ns – 200 ns (Opcode 101 – Dot Product): Matrix coordinates are configured with $A = 8$, $B = 10$, and $C = 2$. The dual-multiplier dot-product structure calculates $(A \times B) + (C \times B) = 100$, producing `32'h00000064` at 200 ns.

200 ns – 220 ns (Opcode 110 – Scaling Shift): The operation applies simultaneous shift protocols, including $A \ll 4$ and $B \gg 2$. The barrel-shift structure produces the expected scaled result of 266, represented as `32'h0000010A`.

220 ns – 260 ns (Opcode 111 – Bitwise Logic): Bitwise logic masks are applied using $(A \text{ XOR } B) \text{ OR } C$. The resulting value is stored in the output register as `32'hFFFFFFF00`.

Project 1: Configurable Saturated DSP Slice Synthesis Performance

Area Footprint: The design consumes 270 Slice LUTs and 96 Slice Registers. The LUT usage is associated with the wide data multiplexers and 32-bit arithmetic barrel-shifter network.

Macro Cell Allocation: The design uses 2 dedicated DSP blocks. Vivado maps the

parallel multiply-accumulate and dot-product structures to dedicated hardware DSP resources.

Power Distribution: The estimated total power is 32.931 W. The internal core consumes 2.453 W, while the physical DSP blocks consume approximately 2.012 W. I/O pad dynamic power accounts for 24.562 W, or approximately 76% of the total reported power.

Timing and Slack Performance: No setup or hold timing failures are reported across the 256 unconstrained endpoints. Because the endpoints are unconstrained, the result should not be interpreted as a measured maximum operating frequency.

Project 2: Configurable Saturated AI Slice Waveform Summary

40 ns – 60 ns (Opcode 00 – 1D Convolution): Input elements are set to [10, 20, -30, 40] and weights to [2, 1, 0, -1]. The structural processing computes $10 \times 2 + 20 \times 1 + (-30) \times 0 + 40 \times (-1) = 40$, producing 32'h00000028 at 60 ns.

60 ns – 80 ns (Opcode 01 – Max Pooling): The vector data streams are routed through parallel comparator structures. The maximum value is selected and written as 32'h00000028 (40) at 80 ns.

80 ns – 100 ns (Opcode 10 – Average Pooling): Three register values representing grid-channel points are summed and divided by four using hardwired right shifts. The scaled output is 10, represented as 32'h0000000A at 100 ns.

100 ns – 120 ns (Opcode 11 – LeakyReLU Activation): The scale registers contain a negative value, and the LeakyReLU path is selected. The hardware performs an arithmetic right shift by 3 positions, implementing multiplication by approximately 0.125 for the negative input and producing a negative fractional result.

120 ns – 160 ns (Opcode 11 – Batch Normalization): When the scale factor is non-zero, the Batch Normalization path is selected. With bias = 1000 and input/scale values as configured in the waveform, the hardware evaluates $Y = (\text{input} \times \text{scale}) + (\text{bias} \ll 16)$, producing the corresponding output at 140 ns.

Project 2: Configurable Saturated AI Slice Synthesis Performance

Area Footprint: The design consumes 174 Slice LUTs and 32 Slice Registers. Its lower LUT and register utilization is associated with mapping computational functions to dedicated hardware resources.

Macro Cell Allocation: The design uses 5 dedicated DSP blocks. The 4-channel 1D

spatial convolution and dot-product operations use parallel DSP macro arrays to support simultaneous arithmetic processing.

Power Distribution: The estimated total power is 35.858 W. Internal logic fabric consumes 1.414 W, while the active high-density DSP macros consume approximately 4.986 W. I/O pad dynamic power accounts for 25.860 W, or approximately 74% of the total reported power.

Timing and Slack Performance: No path violations are reported across the 96 unconstrained endpoints. As with Project 1, the absence of explicit timing constraints means this result confirms structural synthesis behavior rather than establishing a guaranteed maximum clock frequency.

Overall Interpretation

The waveform results demonstrate that both configurable slices correctly execute their assigned operations according to the selected opcode. The DSP Slice supports a broader set of arithmetic, accumulation, shifting, and bitwise operations, while the AI Slice focuses on hardware acceleration for convolution, pooling, LeakyReLU, and normalization functions. The synthesis results show the resource trade-offs between the two architectures: the DSP Slice uses more LUTs and registers, whereas the AI Slice uses more dedicated DSP blocks and a larger parallel data-processing structure. The reported power and timing values reflect the synthesis and analysis conditions used in Vivado, including the fact that the timing endpoints are unconstrained.

Comparative Architectural Trade-off Summary:

The comparative synthesis results establish a clear engineering trade-off between standard digital signal processing and machine learning inference blocks:

- **DSP Block Strategy:** Relies on a generic logic fabric structure. It uses more LUTs and registers to maintain flexible state accumulation loops (pipe_mul, accum_reg) and handle complex bit-level shifts.
- **AI Block Strategy:** Prioritizes parallel execution density. It trades pin footprint and hardware macro cells (5 DSP blocks vs. 2 DSP blocks) for a highly optimized logic footprint, allowing it to process massive input data arrays (196 pins vs. 85 pins) simultaneously without clogging the standard logic routing channels.

Verilog codes:

1.ai_slice.v(design file)

```
`timescale 1ns/1ps

module ai_slice (

    input clk,

    input rst_n,

    input [1:0] opcode,

    input signed [15:0] in0, input signed [15:0] in1,

    input signed [15:0] in2, input signed [15:0] in3,

    input signed [15:0] w0, input signed [15:0] w1,

    input signed [15:0] w2, input signed [15:0] w3,

    input signed [15:0] bn_scale,

    input signed [15:0] bn_bias,

    output reg signed [31:0] Out

);

    wire signed [31:0] comp_max01, comp_max23;

    wire signed [31:0] summation_network;

    wire signed [31:0] leaky_relu_scaled;

    wire signed [31:0] batch_norm_out;

    assign comp_max01 = (in0 > in1) ? in0 : in1;

    assign comp_max23 = (in2 > in3) ? in2 : in3;

    assign summation_network = in0 + in1 + in2 + in3;

    assign leaky_relu_scaled = (in0 < 0) ? {{16{in0}}, in0} >>> 3 : {{16{in0}}, in0};

    assign batch_norm_out = (in0 * bn_scale) + {{16{bn_bias}}, 16'h0};

    always @(posedge clk or negedge rst_n) begin

        if (!rst_n) begin

            Out <= 32'h0;

        end else begin

            case (opcode)

                2'b00: Out <= (in0 * w0) + (in1 * w1) + (in2 * w2) + (in3 * w3);

                2'b01: Out <= (comp_max01 > comp_max23) ? comp_max01 : comp_max23;

                2'b10: Out <= summation_network >>> 2;

                2'b11: Out <= (bn_scale != 0) ? batch_norm_out : leaky_relu_scaled;

                default: Out <= 32'h0;

            endcase

        end

    end

endmodule
```

2.tb_ai_slice.v(testbench file)

```
`timescale 1ns/1ps

module tb_ai_slice;

    reg clk;

    reg rst_n;

    reg [1:0] ai_opcode;

    reg signed [15:0] ai_in0, ai_in1, ai_in2, ai_in3;

    reg signed [15:0] ai_w0, ai_w1, ai_w2, ai_w3;

    reg signed [15:0] ai_scale, ai_bias;

    wire signed [31:0] ai_out;

    ai_slice u_ai_inst (
        .clk(clk), .rst_n(rst_n), .opcode(ai_opcode),
        .in0(ai_in0), .in1(ai_in1), .in2(ai_in2), .in3(ai_in3),
        .w0(ai_w0), .w1(ai_w1), .w2(ai_w2), .w3(ai_w3),
        .bn_scale(ai_scale), .bn_bias(ai_bias), .Out(ai_out)
    );

    always #10 clk = ~clk;

    initial begin
        clk = 0; rst_n = 0;

        ai_opcode = 2'b00; ai_in0 = 0; ai_in1 = 0; ai_in2 = 0; ai_in3 = 0;
        ai_w0 = 0; ai_w1 = 0; ai_w2 = 0; ai_w3 = 0; ai_scale = 0; ai_bias = 0;

        #25 rst_n = 1; #15;

        ai_in0 = 16'd10; ai_in1 = 16'd20; ai_in2 = -16'd30; ai_in3 = 16'd40;
        ai_w0 = 16'd2; ai_w1 = 16'd1; ai_w2 = 16'd0; ai_w3 = -16'd1;
        ai_opcode = 2'b00; #20;
        ai_opcode = 2'b01; #20;
        ai_opcode = 2'b10; #20;
        ai_scale = 16'h0; ai_bias = 16'h0; ai_in0 = -16'd16; ai_opcode = 2'b11; #20;
        ai_in0 = 16'd5; ai_scale = 16'd2; ai_bias = 16'd1000; ai_opcode = 2'b11; #20;
        #40;
        $finish;
    end
endmodule
```

Verilog codes:

1.dsp_slice.v(design file)

```
`timescale 1ns/1ps

module dsp_slice (

    input clk,

    input rst_n,

    input [2:0] opcode,

    input signed [15:0] A,

    input signed [15:0] B,

    input signed [15:0] C,

    output reg signed [31:0] Out

);

    reg signed [31:0] pipe_mul;

    reg signed [31:0] accum_reg;

    always @(posedge clk or negedge rst_n) begin

        if (!rst_n) begin

            pipe_mul <= 32'h0;

            accum_reg <= 32'h0;

            Out <= 32'h0;

        end else begin

            pipe_mul <= A * B;

            case (opcode)

                3'b000: Out <= A + B + C;                // 1. Multi-input Addition

                3'b001: Out <= A - B - C;                // 2. Multi-input Subtraction

                3'b010: Out <= A * B;                    // 3. Vector Element Multiplication

                3'b011: Out <= pipe_mul + Out;            // 4. Multiply-Accumulate (MAC) Matrix Loop

                3'b100: begin                             // 5. Arithmetic Accumulate Vector

                    accum_reg <= accum_reg + A;

                    Out <= accum_reg + A;

                end

                3'b101: Out <= (A * B) + (C * B);        // 6. Dot Product Engine Matrix Row

                3'b110: Out <= (A << 4) + (B >>> 2);    // 7. Vector Scaling Arithmetic Shift (Barrel Shift)

                3'b111: Out <= (A ^ B) | C;              // 8. Bitwise Logic Engine Operations

                default: Out <= 32'h0;

            endcase

        end

    end

endmodule
```

2.tb_dsp_slice.v(testbench file)

```
`timescale 1ns/1ps

module tb_dsp_slice;
reg clk;
reg rst_n;

reg [2:0] dsp_opcode;
reg signed [15:0] dsp_A, dsp_B, dsp_C;

wire signed [31:0] dsp_out;

dsp_slice u_dsp_inst (
.clk(clk),
.rst_n(rst_n),
.opcode(dsp_opcode),
.A(dsp_A),
.B(dsp_B),
.C(dsp_C),
.Out(dsp_out)
);

always #10 clk = ~clk;

initial begin
clk = 0;
rst_n = 0;
dsp_opcode = 3'b000;
dsp_A = 0;
dsp_B = 0;
dsp_C = 0;

#25 rst_n = 1;
#15;

dsp_A = 16'd100; dsp_B = 16'd200; dsp_C = 16'd300; dsp_opcode = 3'b000; #20;

dsp_A = 16'd1000; dsp_B = 16'd400; dsp_C = 16'd100; dsp_opcode = 3'b001; #20;

dsp_A = -16'd25; dsp_B = 16'd4; dsp_opcode = 3'b010; #20;

dsp_opcode = 3'b011; dsp_A = 16'd5; dsp_B = 16'd6; #20;
dsp_A = 16'd10; dsp_B = 16'd2; #20;

dsp_opcode = 3'b100; dsp_A = 16'd15; #20;
dsp_A = 16'd35; #20;

dsp_A = 16'd8; dsp_B = 16'd10; dsp_C = 16'd2; dsp_opcode = 3'b101; #20;

dsp_A = 16'd16; dsp_B = 16'd40; dsp_opcode = 3'b110; #20;

dsp_A = 16'h00FF; dsp_B = 16'h0F0F; dsp_C = 16'hF000; dsp_opcode = 3'b111; #20;

#40;
$finish;
end
endmodule
```